Object Oriented Programming | Variable, Modifier

Variable in Java

A variable is a named memory location used to store data in a Java program.

Example

```
int age = 20;
```

Here:

- `int` → data type
- `age` → variable name
- `20` → value

Easy definition

A variable is a name used to store a value in memory.

Types of Variables in Java

Java variables are divided into **3 types**:

1. Local Variables
2. Instance Variables
3. Class/Static Variables

Local Variable in Java

A local variable is a variable that is declared inside a method, constructor, or block.

It can be used only within that method or block.

- Local variables are declared **inside methods, constructors, or blocks**.
- Local variables are **visible only within** the declared method, constructor, or block.
- **Access modifiers cannot be used** for local variables.

Example

```
public class Test {

    public static void main(String[] args) {

        int age = 20; // Local variable

        System.out.println(age);
    }
}
```

Here, `age` is a local variable because it is declared inside the `main()` method.

Instance Variables in Java

Instance variables are variables declared inside a class but outside methods, constructors, or blocks.

- Each object has its own copy of the instance variables.
- They represent the state or properties of an object.

Example

```
class Student {

    String name;
    int age;

    void display() {
        System.out.println(name);
        System.out.println(age);
    }
}
```

Here, `name` and `age` are instance variables.

Instance Variables in Java

- Instance variables are declared **inside a class**, but **outside a method, constructor, or any block**.
- Instance variables are created when an object is created using the keyword `new` and destroyed when the object is destroyed.
- **Access modifiers can be used** for instance variables.
- Instance variables are **visible to all methods, constructors, and blocks within the class**.

Class/Static Variables in Java

Class/static variables are variables declared inside a class using the `static` keyword and outside methods, constructors, or blocks.

- Only one copy of a static variable is created for the entire class.
- It is shared by all objects of the class.
- It can be accessed using the class name.

Example

```
class Student {

    static String college = "ABC College";
    String name;
}
```

Here, `college` is a static variable because it is declared with the `static` keyword.

Class/Static Variables in Java

- Class variables, also known as **static variables**, are declared using the `static` keyword inside a class but outside a method, constructor, or block.
- Static variables are rarely used except when declared as constants.
- Their visibility is similar to instance variables.
- However, most static variables are declared **public** because they need to be available to users of the class.

Three Types of Variables: Identification

Question

Write a Java program to demonstrate the three types of variables: Local, Instance, and Static variables.

Answer

```
class Student {

    // Instance variable
    String name = "Rahim";

    // Static variable
    static String college = "ABC College";

    void display() {

        // Local variable
        int age = 20;

        System.out.println("Name: " + name);
        System.out.println("College: " + college);
        System.out.println("Age: " + age);
    }

    public static void main(String[] args) {

        Student s = new Student();

        s.display();
    }
}
```

Output

```
Name: Rahim
College: ABC College
Age: 20
```

Identification

- `name` → **Instance Variable**
- `college` → **Static/Class Variable**
- `age` → **Local Variable**

Reason

- `name` → Instance Variable
  Reason: It is declared inside the class but outside the method.

- `college` → Static/Class Variable
  Reason: It is declared with the `static` keyword.
- `age` → Local Variable
  Reason: It is declared inside the `display()` method.

Difference Between Three Types of Variables in Java

Type	Where Declared	Belongs To	Example
Local Variable	Inside a method, constructor, or block	Method/block	`int age = 20;`
Instance Variable	Inside a class but outside methods	Object	`String name;`
Static/Class Variable	Inside a class with `static`, outside methods	Class	`static String college;`

Modifiers in Java

Modifiers are keywords used to change the properties or behavior of classes, variables, methods, and constructors.

Two Main Types of Modifiers

1. Access Modifiers

- `public`
- `private`
- `protected`
- `default`

2. Non-Access Modifiers

- `static`
- `final`
- `abstract`
- `synchronized`
- etc.

Access Modifiers in Java

Access modifiers in Java specify the accessibility (scope) of a data member, method, constructor, or class.

There are **4 types** of Java access modifiers:

1. `private`
2. `default`
3. `protected`
4. `public`

Private Access Modifier in Java

The `private` modifier is used to make a class member accessible only within the same class.

Example

```
class Student {

    private int age = 20;

    void display() {
        System.out.println(age);
    }
}
```

Here, `age` is private, so it can be accessed only inside the `Student` class.

Default Access Modifier in Java

The default access modifier means a member can be accessed within the same package.

Example

```
class Student {

    int age = 20; // default

    void display() {
        System.out.println(age);
    }
}
```

Here, `age` has default access because no access modifier is written.

Protected Access Modifier in Java

The protected access modifier can be accessed:

- Within the same package
- Outside the package through inheritance only

It can be applied to **data members, methods, and constructors**. It cannot be applied to a **class**.

Example

Parent class

```
package pack;

public class A {

    protected int num = 10;

    protected void display() {
        System.out.println("Number: " + num);
    }
}
```

Child class in another package

```
package mypack;

import pack.A;

public class B extends A {

    public static void main(String[] args) {

        B obj = new B();

        System.out.println(obj.num);
        obj.display();
    }
}
```

Output

```
10
Number: 10
```

Why?

`B` is a subclass of `A`, so it can access the protected variable `num` and method `display()` even though they are in different packages.

Public Access Modifier in Java

The `public` access modifier is accessible everywhere.

- It has the widest scope among all access modifiers.
- A public member can be accessed from the same class, same package, different package, and other classes.

Example of Public Access Modifier

```
class Student {

    public int age = 20;

    public void display() {
        System.out.println("Age: " + age);
    }
}

public class Test {

    public static void main(String[] args) {

        Student obj = new Student();

        System.out.println(obj.age);
        obj.display();
    }
}
```

Output

```
20
Age: 20
```

Difference Between Access Modifiers

Modifier	Same Class	Same Package	Different Package	Easy Meaning
`private`	✅	❌	❌	Only same class
default	✅	✅	❌	Only same package
`protected`	✅	✅	✅ through inheritance	Same package + inheritance
`public`	✅	✅	✅	Everywhere

Non-Access Modifiers in Java

1. `static` Modifier

Used for creating class methods and class variables.

2. `final` Modifier

Used for finalizing the implementation of classes, methods, and variables.

3. `abstract` Modifier

Used for creating abstract classes and abstract methods.

4. `synchronized` and `volatile` Modifiers

Used for threads.

Easy Way to Remember

- `static` → Class
- `final` → Cannot be changed
- `abstract` → Incomplete class/method
- `synchronized`, `volatile` → Threads

Difference Between Non-Access Modifiers in Java

Modifier	Main Use	Easy Meaning
`static`	Creates class methods and variables	Belongs to the class
`final`	Finalizes classes, methods, and variables	Cannot be changed/overridden
`abstract`	Creates abstract classes and methods	Used for incomplete implementation
`synchronized` / `volatile`	Used for threads	Helps with thread control and data sharing